

Beginner Level (Warm-ups)

1. Find the largest element

```
# ♠ Without inbuilt
def find_largest(arr):
    largest = arr[0]
    for num in arr:
        if num > largest:
            largest = num
    return largest

# ♠ With inbuilt
def find_largest_inbuilt(arr):
    return max(arr)

print(find_largest([5, 9, 2, 11, 7]))      # 11
print(find_largest_inbuilt([5, 9, 2, 11, 7])) # 11
```

2. Find the smallest element

```
# Without inbuilt
def find_smallest(arr):
    smallest = arr[0]
    for num in arr:
        if num < smallest:
            smallest = num
    return smallest

# With inbuilt
def find_smallest_inbuilt(arr):
    return min(arr)

print(find_smallest([5, 9, 2, 11, 7]))      # 2
print(find_smallest_inbuilt([5, 9, 2, 11, 7])) # 2
```

3. Second largest / second smallest

```
# Without inbuilt
def second_largest(arr):
    largest = second = float('-inf')
    for num in arr:
        if num > largest:
            second = largest
            largest = num
        elif num > second and num != largest:
            second = num
    return second

def second_smallest(arr):
    smallest = second = float('inf')
    for num in arr:
        if num < smallest:
            second = smallest
            smallest = num
        elif num < second and num != smallest:
            second = num
```

```

        return second

# With inbuilt
def second_largest_inbuilt(arr):
    return sorted(set(arr))[-2]

def second_smallest_inbuilt(arr):
    return sorted(set(arr))[1]

print(second_largest([5, 9, 2, 11, 7]))      # 9
print(second_smallest([5, 9, 2, 11, 7]))     # 5
print(second_largest_inbuilt([5, 9, 2, 11, 7])) # 9
print(second_smallest_inbuilt([5, 9, 2, 11, 7])) # 5

```

4. Reverse an array

```

# Without inbuilt
def reverse_array(arr):
    n = len(arr)
    for i in range(n // 2):
        arr[i], arr[n - i - 1] = arr[n - i - 1], arr[i]
    return arr

# With inbuilt
def reverse_array_inbuilt(arr):
    return arr[::-1]

print(reverse_array([1,2,3,4,5]))           # [5,4,3,2,1]
print(reverse_array_inbuilt([1,2,3,4,5]))   # [5,4,3,2,1]

```

5. Rotate array left/right by k

```

# Without inbuilt
def rotate_left(arr, k):
    n = len(arr)
    k %= n
    rotated = []
    for i in range(n):
        rotated.append(arr[(i+k) % n])
    return rotated

def rotate_right(arr, k):
    n = len(arr)
    k %= n
    rotated = []
    for i in range(n):
        rotated.append(arr[(i-k) % n])
    return rotated

# With inbuilt
def rotate_left_inbuilt(arr, k):
    k %= len(arr)
    return arr[k:] + arr[:k]

def rotate_right_inbuilt(arr, k):
    k %= len(arr)
    return arr[-k:] + arr[:-k]

print(rotate_left([1,2,3,4,5], 2))          # [3,4,5,1,2]
print(rotate_right([1,2,3,4,5], 2))         # [4,5,1,2,3]

```

```
print(rotate_left_inbuilt([1,2,3,4,5], 2)) # [3,4,5,1,2]
print(rotate_right_inbuilt([1,2,3,4,5], 2)) # [4,5,1,2,3]
```

6. Sum of elements

```
# Without inbuilt
def sum_array(arr):
    total = 0
    for num in arr:
        total += num
    return total

# With inbuilt
def sum_array_inbuilt(arr):
    return sum(arr)

print(sum_array([1,2,3,4,5])) # 15
print(sum_array_inbuilt([1,2,3,4,5])) # 15
```

7. Average of elements

```
# Without inbuilt
def average_array(arr):
    total = 0
    count = 0
    for num in arr:
        total += num
        count += 1
    return total / count

# With inbuilt
def average_array_inbuilt(arr):
    return sum(arr) / len(arr)

print(average_array([1,2,3,4,5])) # 3.0
print(average_array_inbuilt([1,2,3,4,5])) # 3.0
```

8. Count even and odd numbers

```
# Without inbuilt
def count_even_odd(arr):
    even = odd = 0
    for num in arr:
        if num % 2 == 0:
            even += 1
        else:
            odd += 1
    return even, odd

# With inbuilt
def count_even_odd_inbuilt(arr):
    even = len([x for x in arr if x % 2 == 0])
    odd = len(arr) - even
    return even, odd

print(count_even_odd([1,2,3,4,5,6])) # (3,3)
print(count_even_odd_inbuilt([1,2,3,4,5,6])) # (3,3)
```

9. Check if array is sorted

```
# Without inbuilt
def is_sorted(arr):
    ascending = descending = True
    for i in range(1, len(arr)):
        if arr[i] < arr[i-1]:
            ascending = False
        if arr[i] > arr[i-1]:
            descending = False
    if ascending: return "Ascending"
    if descending: return "Descending"
    return "Not Sorted"

# With inbuilt
def is_sorted_inbuilt(arr):
    if arr == sorted(arr):
        return "Ascending"
    elif arr == sorted(arr, reverse=True):
        return "Descending"
    else:
        return "Not Sorted"

print(is_sorted([1,2,3,4]))          # Ascending
print(is_sorted([9,7,5,2]))          # Descending
print(is_sorted([3,1,4,2]))          # Not Sorted
print(is_sorted_inbuilt([1,2,3,4]))  # Ascending
```

10. Merge two sorted arrays

```
# Without inbuilt (two-pointer technique)
def merge_sorted(arr1, arr2):
    i = j = 0
    merged = []
    while i < len(arr1) and j < len(arr2):
        if arr1[i] < arr2[j]:
            merged.append(arr1[i])
            i += 1
        else:
            merged.append(arr2[j])
            j += 1
    while i < len(arr1):
        merged.append(arr1[i])
        i += 1
    while j < len(arr2):
        merged.append(arr2[j])
        j += 1
    return merged

# With inbuilt
def merge_sorted_inbuilt(arr1, arr2):
    return sorted(arr1 + arr2)

print(merge_sorted([1,3,5], [2,4,6]))          # [1,2,3,4,5,6]
print(merge_sorted_inbuilt([1,3,5], [2,4,6]))  # [1,2,3,4,5,6]
```

Searching & Sorting

11. Linear Search

```
# Without inbuilt
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# With inbuilt
def linear_search_inbuilt(arr, target):
    return arr.index(target) if target in arr else -1

print(linear_search([2, 4, 6, 8], 6))          # 2
print(linear_search_inbuilt([2, 4, 6, 8], 6))  # 2
```

12. Binary Search (Iterative + Recursive)

```
# Without inbuilt - Iterative
def binary_search_iter(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low+high)//2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid+1
        else:
            high = mid-1
    return -1

# Without inbuilt - Recursive
def binary_search_rec(arr, target, low, high):
    if low > high:
        return -1
    mid = (low+high)//2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_rec(arr, target, mid+1, high)
    else:
        return binary_search_rec(arr, target, low, mid-1)

# With inbuilt
import bisect
def binary_search_inbuilt(arr, target):
    idx = bisect.bisect_left(arr, target)
    return idx if idx < len(arr) and arr[idx] == target else -1

print(binary_search_iter([1,3,5,7,9], 7))      # 3
print(binary_search_rec([1,3,5,7,9], 7, 0, 4)) # 3
print(binary_search_inbuilt([1,3,5,7,9], 7))   # 3
```

13. First and Last Occurrence

```
# Without inbuilt
def first_last_occurrence(arr, target):
    first, last = -1, -1
    for i in range(len(arr)):
```

```

        if arr[i] == target:
            if first == -1:
                first = i
            last = i
    return first, last

# With inbuilt
def first_last_inbuilt(arr, target):
    if target not in arr:
        return -1, -1
    return arr.index(target), len(arr) - 1 - arr[::-1].index(target)

print(first_last_occurrence([1,2,3,2,4,2], 2))    # (1,5)
print(first_last_inbuilt([1,2,3,2,4,2], 2))      # (1,5)

```

14. Count Occurrences

```

# Without inbuilt
def count_occurrence(arr, target):
    count = 0
    for num in arr:
        if num == target:
            count += 1
    return count

# With inbuilt
def count_occurrence_inbuilt(arr, target):
    return arr.count(target)

print(count_occurrence([1,2,3,2,2,4], 2))        # 3
print(count_occurrence_inbuilt([1,2,3,2,2,4], 2)) # 3

```

15. Missing Number in 1..n

```

# Without inbuilt
def missing_number(arr, n):
    expected_sum = n * (n+1) // 2
    actual_sum = 0
    for num in arr:
        actual_sum += num
    return expected_sum - actual_sum

# With inbuilt
def missing_number_inbuilt(arr, n):
    return sum(range(1, n+1)) - sum(arr)

print(missing_number([1,2,4,5], 5))              # 3
print(missing_number_inbuilt([1,2,4,5], 5))      # 3

```

16. Find Duplicate Numbers

```

# Without inbuilt
def find_duplicates(arr):
    seen, duplicates = [], []
    for num in arr:
        if num in seen and num not in duplicates:
            duplicates.append(num)
        else:
            seen.append(num)
    return duplicates

```

```
# With inbuilt (set logic)
def find_duplicates_inbuilt(arr):
    return list({x for x in arr if arr.count(x) > 1})

print(find_duplicates([1,2,3,2,4,1,5,3])) # [2,1,3]
print(find_duplicates_inbuilt([1,2,3,2,4,1,5,3])) # [1,2,3]
```

17. All Pairs with Sum = k

```
# Without inbuilt
def find_pairs(arr, k):
    pairs = []
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] + arr[j] == k:
                pairs.append((arr[i], arr[j]))
    return pairs

# With inbuilt (set trick)
def find_pairs_inbuilt(arr, k):
    seen, pairs = set(), set()
    for num in arr:
        if k - num in seen:
            pairs.add((min(num, k-num), max(num, k-num)))
        seen.add(num)
    return list(pairs)

print(find_pairs([1,2,3,4,5,6], 7)) # [(1,6), (2,5), (3,4)]
print(find_pairs_inbuilt([1,2,3,4,5,6], 7)) # [(1,6), (2,5), (3,4)]
```

18. Triplets with Sum = k

```
# Without inbuilt
def find_triplets(arr, k):
    n = len(arr)
    triplets = []
    for i in range(n):
        for j in range(i+1, n):
            for l in range(j+1, n):
                if arr[i] + arr[j] + arr[l] == k:
                    triplets.append((arr[i], arr[j], arr[l]))
    return triplets

# With inbuilt (sorting + two-pointer)
def find_triplets_inbuilt(arr, k):
    arr.sort()
    triplets = []
    n = len(arr)
    for i in range(n-2):
        left, right = i+1, n-1
        while left < right:
            s = arr[i] + arr[left] + arr[right]
            if s == k:
                triplets.append((arr[i], arr[left], arr[right]))
                left += 1
                right -= 1
            elif s < k:
                left += 1
            else:
                right -= 1
```

```

        right -= 1
    return triplets

print(find_triplets([1,2,3,4,5], 9))      # [(1,3,5), (2,3,4)]
print(find_triplets_inbuilt([1,2,3,4,5], 9))# [(1,3,5), (2,3,4)]

```

19. Bubble / Selection / Insertion Sort

```

# Bubble Sort (without inbuilt)
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

# Selection Sort
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

# Insertion Sort
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i-1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr

# With inbuilt
def sort_inbuilt(arr):
    return sorted(arr)

print(bubble_sort([5,3,1,4,2]))      # [1,2,3,4,5]
print(selection_sort([5,3,1,4,2]))  # [1,2,3,4,5]
print(insertion_sort([5,3,1,4,2]))  # [1,2,3,4,5]
print(sort_inbuilt([5,3,1,4,2]))    # [1,2,3,4,5]

```

20. Merge Sort / Quick Sort

```

# Merge Sort
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):

```



```
merged, i, j = [], 0, 0
while i < len(left) and j < len(right):
    if left[i] < right[j]:
        merged.append(left[i]); i+=1
    else:
        merged.append(right[j]); j+=1
merged.extend(left[i:])
merged.extend(right[j:])
return merged

# Quick Sort
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    less = [x for x in arr[1:] if x <= pivot]
    greater = [x for x in arr[1:] if x > pivot]
    return quick_sort(less) + [pivot] + quick_sort(greater)

# With inbuilt
def sort_inbuilt(arr):
    return sorted(arr)

print(merge_sort([5,3,1,4,2])) # [1,2,3,4,5]
print(quick_sort([5,3,1,4,2])) # [1,2,3,4,5]
print(sort_inbuilt([5,3,1,4,2])) # [1,2,3,4,5]
```

Subarray / Subsequence Problems (21–28)

✓ Subarray / Subsequence Problems (21–28)

21. Maximum Subarray Sum (Kadane's Algorithm)

Without inbuilt

```
def max_subarray_sum(arr):  
    max_sum = arr[0]  
    curr_sum = arr[0]  
    for i in range(1, len(arr)):  
        curr_sum = arr[i] if arr[i] > curr_sum + arr[i] else curr_sum + arr[i]  
        max_sum = curr_sum if curr_sum > max_sum else max_sum  
    return max_sum
```

With inbuilt

```
def max_subarray_sum_inbuilt(arr):  
    max_sum = arr[0]  
    curr_sum = arr[0]  
    for i in range(1, len(arr)):  
        curr_sum = max(arr[i], curr_sum + arr[i])  
        max_sum = max(max_sum, curr_sum)  
    return max_sum
```

```
print(max_subarray_sum([-2,1,-3,4,-1,2,1,-5,4]))    # 6
```

```
print(max_subarray_sum_inbuilt([-2,1,-3,4,-1,2,1,-5,4])) # 6
```

22. Subarray with Given Sum = `k`

Without inbuilt (brute force $O(n^2)$)

```
def subarray_sum(arr, k):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        s = 0
```

```
        for j in range(i, n):
```

```
            s += arr[j]
```

```
            if s == k:
```

```
                return arr[i:j+1]
```

```
    return []
```

With inbuilt (prefix sum + dict)

```
def subarray_sum_inbuilt(arr, k):
```

```
    prefix_sum, seen = 0, {0: -1}
```

```
    for i, num in enumerate(arr):
```

```
        prefix_sum += num
```

```
        if prefix_sum - k in seen:
```

```
            return arr[seen[prefix_sum-k]+1:i+1]
```

```
        seen[prefix_sum] = i
```

```
    return []
```

```
print(subarray_sum([1,2,3,7,5], 12))    # [2,3,7]
```

```
print(subarray_sum_inbuilt([1,2,3,7,5], 12)) # [2,3,7]
```

23. Longest Subarray with Sum = `k`

Without inbuilt

```
def longest_subarray_sum(arr, k):
```

```
    n = len(arr)
```

```

max_len = 0
for i in range(n):
    s = 0
    for j in range(i, n):
        s += arr[j]
        if s == k and (j-i+1) > max_len:
            max_len = j-i+1
return max_len

```

With inbuilt (prefix sum + hashmap)

```

def longest_subarray_sum_inbuilt(arr, k):
    prefix_sum, seen, max_len = 0, {}, 0
    for i, num in enumerate(arr):
        prefix_sum += num
        if prefix_sum == k:
            max_len = i+1
        if prefix_sum-k in seen:
            max_len = max(max_len, i - seen[prefix_sum-k])
        if prefix_sum not in seen:
            seen[prefix_sum] = i
    return max_len

```

```

print(longest_subarray_sum([1,2,3,1,1,1,2], 3))    # 2
print(longest_subarray_sum_inbuilt([1,2,3,1,1,1,2], 3)) # 2

```

24. Longest Subarray with Equal 0s and 1s

Without inbuilt (brute force $O(n^2)$)

```

def longest_equal_01(arr):
    n = len(arr)

```

```

max_len = 0
for i in range(n):
    zeros, ones = 0, 0
    for j in range(i, n):
        if arr[j] == 0:
            zeros += 1
        else:
            ones += 1
        if zeros == ones and (j-i+1) > max_len:
            max_len = j-i+1
    return max_len

```

With inbuilt (prefix sum + hashmap)

```

def longest_equal_01_inbuilt(arr):
    s, seen, max_len = 0, {0: -1}, 0
    for i, num in enumerate(arr):
        s += 1 if num == 1 else -1
        if s in seen:
            max_len = max(max_len, i - seen[s])
        else:
            seen[s] = i
    return max_len

```

```

print(longest_equal_01([0,1,0,1,1,0,0]))    # 6
print(longest_equal_01_inbuilt([0,1,0,1,1,0,0])) # 6

```

25. Maximum Product Subarray

Without inbuilt

```

def max_product_subarray(arr):

```

```

max_prod = min_prod = result = arr[0]
for i in range(1, len(arr)):
    num = arr[i]
    temp_max = max_prod
    max_prod = num if num > max(num, min_prod*num) else max(num, min_prod*num)
    min_prod = num if num < min(num, temp_max*num) else min(num, temp_max*num)
    result = result if result > max_prod else max_prod
return result

```

With inbuilt

```

def max_product_subarray_inbuilt(arr):
    max_prod = min_prod = result = arr[0]
    for num in arr[1:]:
        choices = (num, max_prod*num, min_prod*num)
        max_prod = max(choices)
        min_prod = min(choices)
        result = max(result, max_prod)
    return result

```

```

print(max_product_subarray([2,3,-2,4]))    # 6
print(max_product_subarray_inbuilt([2,3,-2,4])) # 6

```

26. Count Subarrays with Sum = `k`

Without inbuilt (brute force $O(n^2)$)

```

def count_subarrays_sum(arr, k):
    n = len(arr)
    count = 0
    for i in range(n):
        s = 0

```

```

    for j in range(i, n):
        s += arr[j]
        if s == k:
            count += 1
    return count

```

With inbuilt (prefix sum + hashmap)

```

def count_subarrays_sum_inbuilt(arr, k):
    prefix_sum, seen, count = 0, {0:1}, 0
    for num in arr:
        prefix_sum += num
        count += seen.get(prefix_sum-k, 0)
        seen[prefix_sum] = seen.get(prefix_sum, 0) + 1
    return count

```

```

print(count_subarrays_sum([1,2,3], 3))    # 2 ([1,2], [3])

```

```

print(count_subarrays_sum_inbuilt([1,2,3], 3)) # 2

```

27. Minimum Length Subarray with Sum $\geq k$

Without inbuilt (brute force $O(n^2)$)

```

def min_len_subarray(arr, k):
    n = len(arr)
    min_len = n+1
    for i in range(n):
        s = 0
        for j in range(i, n):
            s += arr[j]
            if s >= k and (j-i+1) < min_len:
                min_len = j-i+1

```

```

        break
    return 0 if min_len == n+1 else min_len

# With inbuilt (sliding window O(n))
def min_len_subarray_inbuilt(arr, k):
    left, s, min_len = 0, 0, float('inf')
    for right in range(len(arr)):
        s += arr[right]
        while s >= k:
            min_len = min(min_len, right-left+1)
            s -= arr[left]
            left += 1
    return 0 if min_len == float('inf') else min_len

print(min_len_subarray([2,3,1,2,4,3], 7))    # 2 ([4,3])
print(min_len_subarray_inbuilt([2,3,1,2,4,3], 7)) # 2

```

28. Longest Increasing Subarray

```

# Without inbuilt
def longest_increasing_subarray(arr):
    max_len = curr_len = 1
    for i in range(1, len(arr)):
        if arr[i] > arr[i-1]:
            curr_len += 1
            if curr_len > max_len:
                max_len = curr_len
        else:
            curr_len = 1
    return max_len

```


With inbuilt

```
def longest_increasing_subarray_inbuilt(arr):
```

```
    from itertools import groupby
```

```
    max_len, curr_len = 1, 1
```

```
    for i in range(1, len(arr)):
```

```
        curr_len = curr_len+1 if arr[i] > arr[i-1] else 1
```

```
        max_len = max(max_len, curr_len)
```

```
    return max_len
```

```
print(longest_increasing_subarray([1,2,2,3,4,1,2,3]))    # 3
```

```
print(longest_increasing_subarray_inbuilt([1,2,2,3,4,1,2,3])) # 3
```

Matrix (2D Array) Questions

✓ Matrix Problems (29–35)

29. Search in a Row-wise & Column-wise Sorted Matrix

Without inbuilt (staircase search $O(m+n)$)

```
def search_sorted_matrix(matrix, target):
```

```
    rows, cols = len(matrix), len(matrix[0])
```

```
    i, j = 0, cols - 1
```

```
    while i < rows and j >= 0:
```

```
        if matrix[i][j] == target:
```

```
            return True
```

```
        elif matrix[i][j] > target:
```

```
            j -= 1
```

```
        else:
```

```
            i += 1
```

```
    return False
```

With inbuilt (flatten + in operator)

```
def search_sorted_matrix_inbuilt(matrix, target):
```

```
    return any(target in row for row in matrix)
```

```
mat = [[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]]
```

```
print(search_sorted_matrix(mat, 5))    # True
```

```
print(search_sorted_matrix_inbuilt(mat, 5)) # True
```

30. Rotate Matrix by 90 Degrees

Without inbuilt (transpose + reverse rows)

```
def rotate_matrix(matrix):  
    n = len(matrix)  
  
    # Transpose  
    for i in range(n):  
        for j in range(i, n):  
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]  
  
    # Reverse rows  
    for i in range(n):  
        for j in range(n//2):  
            matrix[i][j], matrix[i][n-j-1] = matrix[i][n-j-1], matrix[i][j]  
  
    return matrix
```

With inbuilt (zip + slicing)

```
def rotate_matrix_inbuilt(matrix):  
    return [list(row) for row in zip(*matrix[::-1])]
```

```
mat = [[1,2,3],[4,5,6],[7,8,9]]
```

```
print(rotate_matrix([row[:] for row in mat]))    # [[7,4,1],[8,5,2],[9,6,3]]
```

```
print(rotate_matrix_inbuilt(mat))                # [[7,4,1],[8,5,2],[9,6,3]]
```

31. Print Matrix in Spiral Order

Without inbuilt

```
def spiral_order(matrix):  
    res = []  
  
    top, bottom, left, right = 0, len(matrix)-1, 0, len(matrix[0])-1  
  
    while top <= bottom and left <= right:
```

```

for i in range(left, right+1): res.append(matrix[top][i])
top += 1
for i in range(top, bottom+1): res.append(matrix[i][right])
right -= 1
if top <= bottom:
    for i in range(right, left-1, -1): res.append(matrix[bottom][i])
    bottom -= 1
if left <= right:
    for i in range(bottom, top-1, -1): res.append(matrix[i][left])
    left += 1
return res

```

With inbuilt (flatten trick + slicing loops)

```

def spiral_order_inbuilt(matrix):
    res = []
    while matrix:
        res += matrix.pop(0)
        if matrix and matrix[0]:
            for row in matrix: res.append(row.pop())
        if matrix:
            res += matrix.pop()[::-1]
        if matrix and matrix[0]:
            for row in matrix[::-1]: res.append(row.pop(0))
    return res

```

```

mat = [[1,2,3],[4,5,6],[7,8,9]]
print(spiral_order([row[:] for row in mat]))    # [1,2,3,6,9,8,7,4,5]
print(spiral_order_inbuilt([row[:] for row in mat]))# [1,2,3,6,9,8,7,4,5]

```

32. Transpose of a Matrix

Without inbuilt

```
def transpose(matrix):  
    rows, cols = len(matrix), len(matrix[0])  
    transposed = [[0]*rows for _ in range(cols)]  
    for i in range(rows):  
        for j in range(cols):  
            transposed[j][i] = matrix[i][j]  
    return transposed
```

With inbuilt (zip)

```
def transpose_inbuilt(matrix):  
    return [list(row) for row in zip(*matrix)]
```

```
mat = [[1,2,3],[4,5,6]]  
print(transpose(mat))      # [[1,4],[2,5],[3,6]]  
print(transpose_inbuilt(mat))# [[1,4],[2,5],[3,6]]
```

33. Maximum Sum Submatrix

☞ This is the 2D Kadane's Algorithm problem.

Without inbuilt (2D Kadane $O(n^3)$)

```
def max_sum_submatrix(matrix):  
    rows, cols = len(matrix), len(matrix[0])  
    max_sum = float('-inf')  
    for left in range(cols):  
        temp = [0]*rows  
        for right in range(left, cols):  
            for i in range(rows):
```

```

        temp[i] += matrix[i][right]
# Apply 1D Kadane
curr_sum = temp[0]
best = temp[0]
for i in range(1, rows):
    curr_sum = temp[i] if temp[i] > curr_sum+temp[i] else curr_sum+temp[i]
    best = best if best > curr_sum else curr_sum
max_sum = max(max_sum, best)
return max_sum

```

With inbuilt (reuse sum() + max() for Kadane)

```

def max_sum_submatrix_inbuilt(matrix):
    rows, cols = len(matrix), len(matrix[0])
    max_sum = float('-inf')
    for left in range(cols):
        temp = [0]*rows
        for right in range(left, cols):
            temp = [temp[i]+matrix[i][right] for i in range(rows)]
            curr_sum = best = temp[0]
            for num in temp[1:]:
                curr_sum = max(num, curr_sum+num)
                best = max(best, curr_sum)
            max_sum = max(max_sum, best)
    return max_sum

```

```

mat = [[1,-2,-1,4],[2,3,4,-5],[-3,4,5,6]]
print(max_sum_submatrix(mat))    # 18
print(max_sum_submatrix_inbuilt(mat))# 18

```

34. Diagonal Sum of a Square Matrix

Without inbuilt

```
def diagonal_sum(matrix):  
    n = len(matrix)  
    total = 0  
    for i in range(n):  
        total += matrix[i][i] # primary diagonal  
        if i != n-i-1:  
            total += matrix[i][n-i-1] # secondary diagonal  
    return total
```

With inbuilt

```
def diagonal_sum_inbuilt(matrix):  
    n = len(matrix)  
    return sum(matrix[i][i] + (matrix[i][n-i-1] if i != n-i-1 else 0) for i in range(n))
```

```
mat = [[1,2,3],[4,5,6],[7,8,9]]  
print(diagonal_sum(mat))    # 25  
print(diagonal_sum_inbuilt(mat))# 25
```

35. Set Row & Column to Zero if an Element is Zero

Without inbuilt

```
def set_zeroes(matrix):  
    rows, cols = len(matrix), len(matrix[0])  
    row_set, col_set = [], []  
    for i in range(rows):  
        for j in range(cols):  
            if matrix[i][j] == 0:  
                row_set.append(i)
```

```

        col_set.append(j)
    for i in range(rows):
        for j in range(cols):
            if i in row_set or j in col_set:
                matrix[i][j] = 0
    return matrix

```

With inbuilt (set comprehension)

```

def set_zeroes_inbuilt(matrix):
    rows, cols = len(matrix), len(matrix[0])
    zero_rows = {i for i in range(rows) for j in range(cols) if matrix[i][j]==0}
    zero_cols = {j for i in range(rows) for j in range(cols) if matrix[i][j]==0}
    for i in range(rows):
        for j in range(cols):
            if i in zero_rows or j in zero_cols:
                matrix[i][j] = 0
    return matrix

```

```
mat = [[1,2,3],[4,0,6],[7,8,9]]
```

```
print(set_zeroes([row[:] for row in mat]))    # [[1,0,3],[0,0,0],[7,0,9]]
```

```
print(set_zeroes_inbuilt([row[:] for row in mat]))# [[1,0,3],[0,0,0],[7,0,9]]
```


Advanced Array Problem

✓ Advanced Array Problems (36–50)

36. Majority Element ($> n/2$ times)

Without inbuilt (Boyer-Moore Voting Algorithm)

```
def majority_element(arr):  
    count, candidate = 0, None  
    for num in arr:  
        if count == 0:  
            candidate = num  
        count += 1 if num == candidate else -1  
    return candidate
```

With inbuilt (collections.Counter)

```
from collections import Counter  
def majority_element_inbuilt(arr):  
    return Counter(arr).most_common(1)[0][0]
```

```
print(majority_element([3,3,4,2,3,3,5]))    # 3  
print(majority_element_inbuilt([3,3,4,2,3,3,5])) # 3
```

37. Elements Occurring More Than $n/3$ Times

Without inbuilt (Extended Boyer-Moore)

```
def majority_element_n3(arr):
```

```
cand1, cand2, count1, count2 = None, None, 0, 0
```

```
for num in arr:
```

```
    if num == cand1:
```

```
        count1 += 1
```

```
    elif num == cand2:
```

```
        count2 += 1
```

```
    elif count1 == 0:
```

```
        cand1, count1 = num, 1
```

```
    elif count2 == 0:
```

```
        cand2, count2 = num, 1
```

```
    else:
```

```
        count1 -= 1
```

```
        count2 -= 1
```

```
result = []
```

```
for cand in [cand1, cand2]:
```

```
    if arr.count(cand) > len(arr)//3:
```

```
        result.append(cand)
```

```
return result
```

```
# With inbuilt
```

```
def majority_element_n3_inbuilt(arr):
```

```
    from collections import Counter
```

```
    freq = Counter(arr)
```

```
    return [num for num, cnt in freq.items() if cnt > len(arr)//3]
```

```
print(majority_element_n3([1,2,3,1,2,1,1]))    # [1]
```

```
print(majority_element_n3_inbuilt([1,2,3,1,2,1,1])) # [1]
```

```
## 38. Equilibrium Index
```

Without inbuilt

```
def equilibrium_index(arr):  
    total = 0  
    for num in arr:  
        total += num  
    left_sum = 0  
    for i, num in enumerate(arr):  
        if left_sum == total - left_sum - num:  
            return i  
        left_sum += num  
    return -1
```

With inbuilt

```
def equilibrium_index_inbuilt(arr):  
    total = sum(arr)  
    left_sum = 0  
    for i, num in enumerate(arr):  
        if left_sum == total - left_sum - num:  
            return i  
        left_sum += num  
    return -1
```

```
print(equilibrium_index([1,3,5,2,2]))    # 2
```

```
print(equilibrium_index_inbuilt([1,3,5,2,2])) # 2
```

39. Trapping Rainwater

Without inbuilt

```
def trap_rainwater(arr):  
    n = len(arr)
```

```

left_max, right_max = [0]*n, [0]*n
left_max[0] = arr[0]
for i in range(1,n):
    left_max[i] = arr[i] if arr[i]>left_max[i-1] else left_max[i-1]
right_max[-1] = arr[-1]
for i in range(n-2,-1,-1):
    right_max[i] = arr[i] if arr[i]>right_max[i+1] else right_max[i+1]
water = 0
for i in range(n):
    water += min(left_max[i], right_max[i]) - arr[i]
return water

```

With inbuilt

```

def trap_rainwater_inbuilt(arr):
    n = len(arr)
    left_max = [max(arr[:i+1]) for i in range(n)]
    right_max = [max(arr[i:]) for i in range(n)]
    return sum(min(left_max[i], right_max[i]) - arr[i] for i in range(n))

```

```

print(trap_rainwater([0,1,0,2,1,0,1,3,2,1,2,1]))    # 6
print(trap_rainwater_inbuilt([0,1,0,2,1,0,1,3,2,1,2,1])) # 6

```

40. Container With Most Water

Without inbuilt (two pointer)

```

def max_area(height):
    l, r = 0, len(height)-1
    area = 0
    while l < r:
        area = area if area > min(height[l],height[r])*(r-l) else min(height[l],height[r])*(r-l)

```

```

    if height[l] < height[r]:
        l += 1
    else:
        r -= 1
return area

```

With inbuilt (same but with max())

```

def max_area_inbuilt(height):
    l, r, ans = 0, len(height)-1, 0
    while l < r:
        ans = max(ans, min(height[l], height[r])*(r-l))
        if height[l] < height[r]:
            l += 1
        else:
            r -= 1
    return ans

```

```

print(max_area([1,8,6,2,5,4,8,3,7]))    # 49

```

```

print(max_area_inbuilt([1,8,6,2,5,4,8,3,7])) # 49

```

41. Maximum Difference (j > i)

Without inbuilt

```

def max_diff(arr):
    min_val, max_diff = arr[0], arr[1]-arr[0]
    for i in range(1, len(arr)):
        if arr[i]-min_val > max_diff:

```

```
        max_diff = arr[i]-min_val
    if arr[i] < min_val:
        min_val = arr[i]
    return max_diff
```

With inbuilt

```
def max_diff_inbuilt(arr):
    min_val, max_diff = arr[0], float('-inf')
    for num in arr[1:]:
        max_diff = max(max_diff, num - min_val)
        min_val = min(min_val, num)
    return max_diff
```

```
print(max_diff([2,3,10,6,4,8,1]))    # 8
print(max_diff_inbuilt([2,3,10,6,4,8,1])) # 8
```

42. Next Greater Element

Without inbuilt (stack O(n))

```
def next_greater(arr):
    n = len(arr)
    res = [-1]*n
    stack = []
    for i in range(n-1,-1,-1):
        while stack and stack[-1] <= arr[i]:
            stack.pop()
        if stack:
            res[i] = stack[-1]
        stack.append(arr[i])
    return res
```

With inbuilt (using enumerate and list comp)

```
def next_greater_inbuilt(arr):
```

```
    res, stack = [-1]*len(arr), []
```

```
    for i in range(len(arr)-1,-1,-1):
```

```
        while stack and stack[-1] <= arr[i]: stack.pop()
```

```
        res[i] = stack[-1] if stack else -1
```

```
        stack.append(arr[i])
```

```
    return res
```

```
print(next_greater([4,5,2,25]))    # [5,25,25,-1]
```

```
print(next_greater_inbuilt([4,5,2,25])) # [5,25,25,-1]
```

43. Next Smaller Element

Without inbuilt

```
def next_smaller(arr):
```

```
    n = len(arr)
```

```
    res = [-1]*n
```

```
    stack = []
```

```
    for i in range(n-1,-1,-1):
```

```
        while stack and stack[-1] >= arr[i]:
```

```
            stack.pop()
```

```
        if stack:
```

```
            res[i] = stack[-1]
```

```
            stack.append(arr[i])
```

```
    return res
```

With inbuilt

```
def next_smaller_inbuilt(arr):
```

```

res, stack = [-1]*len(arr), []
for i in range(len(arr)-1,-1,-1):
    while stack and stack[-1] >= arr[i]: stack.pop()
    res[i] = stack[-1] if stack else -1
    stack.append(arr[i])
return res

print(next_smaller([4,5,2,10,8]))    # [2,2,-1,8,-1]
print(next_smaller_inbuilt([4,5,2,10,8])) # [2,2,-1,8,-1]

```

44. Stock Buy and Sell

* **I (single transaction)**

* **II (multiple transactions)**

* **Cooldown (DP)**

I: Single transaction

```

def max_profit_one(arr):
    min_price, max_profit = arr[0], 0
    for price in arr:
        if price < min_price: min_price = price
        if price-min_price > max_profit: max_profit = price-min_price
    return max_profit

```

II: Multiple transactions

```

def max_profit_two(arr):
    profit = 0
    for i in range(1,len(arr)):
        if arr[i] > arr[i-1]:
            profit += arr[i]-arr[i-1]

```



```
return profit
```

```
# III: With cooldown (DP)
```

```
def max_profit_cooldown(prices):
```

```
    if not prices: return 0
```

```
    n = len(prices)
```

```
    hold, sold, rest = -prices[0], 0, 0
```

```
    for i in range(1,n):
```

```
        hold = max(hold, rest - prices[i])
```

```
        rest = max(rest, sold)
```

```
        sold = hold + prices[i]
```

```
    return max(sold, rest)
```

```
print(max_profit_one([7,1,5,3,6,4]))    # 5
```

```
print(max_profit_two([7,1,5,3,6,4]))    # 7
```

```
print(max_profit_cooldown([1,2,3,0,2])) # 3
```

```
## 45. Kth Smallest / Largest
```

```
# Without inbuilt (Quickselect)
```

```
def partition(arr, low, high):
```

```
    pivot = arr[high]
```

```
    i = low
```

```
    for j in range(low, high):
```

```
        if arr[j] <= pivot:
```

```
            arr[i], arr[j] = arr[j], arr[i]
```

```
            i += 1
```

```
    arr[i], arr[high] = arr[high], arr[i]
```

```
    return i
```

```
def quickselect(arr, low, high, k):
```

```

if low <= high:
    pi = partition(arr, low, high)
    if pi == k: return arr[pi]
    elif pi < k: return quickselect(arr, pi+1, high, k)
    else: return quickselect(arr, low, pi-1, k)

```

```

def kth_smallest(arr, k):
    return quickselect(arr[:], 0, len(arr)-1, k-1)

```

With inbuilt (heapq)

```

import heapq
def kth_smallest_inbuilt(arr, k):
    return heapq.nsmallest(k, arr)[-1]
def kth_largest_inbuilt(arr, k):
    return heapq.nlargest(k, arr)[-1]

```

```

print(kth_smallest([7,10,4,3,20,15], 3))    # 7
print(kth_smallest_inbuilt([7,10,4,3,20,15], 3)) # 7
print(kth_largest_inbuilt([7,10,4,3,20,15], 3)) # 10

```

46. Merge Intervals

Without inbuilt

```

def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for i in range(1, len(intervals)):
        if intervals[i][0] <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], intervals[i][1])
        else:

```

```
        merged.append(intervals[i])
    return merged
```

With inbuilt

```
def merge_intervals_inbuilt(intervals):
```

```
    intervals.sort()
```

```
    res = []
```

```
    for s,e in intervals:
```

```
        if res and s <= res[-1][1]:
```

```
            res[-1][1] = max(res[-1][1], e)
```

```
        else:
```

```
            res.append([s,e])
```

```
    return res
```

```
print(merge_intervals([[1,3],[2,6],[8,10],[15,18]]))    # [[1,6],[8,10],[15,18]]
```

```
print(merge_intervals_inbuilt([[1,3],[2,6],[8,10],[15,18]])) # [[1,6],[8,10],[15,18]]
```

47. Insert Interval

Without inbuilt

```
def insert_interval(intervals, new_interval):
```

```
    res = []
```

```
    for i in range(len(intervals)):
```

```
        if intervals[i][1] < new_interval[0]:
```

```
            res.append(intervals[i])
```

```
        elif intervals[i][0] > new_interval[1]:
```

```
            res.append(new_interval)
```

```
            return res + intervals[i:]
```

```
        else:
```

```
            new_interval[0] = min(new_interval[0], intervals[i][0])
```

```

        new_interval[1] = max(new_interval[1], intervals[i][1])
    res.append(new_interval)
    return res

```

With inbuilt (same logic but cleaner)

```
def insert_interval_inbuilt(intervals, new):
```

```

    res = []
    for s,e in intervals:
        if e < new[0]: res.append([s,e])
        elif s > new[1]:
            res.append(new); new=[s,e]
        else:
            new[0] = min(new[0], s); new[1] = max(new[1], e)
    res.append(new)
    return res

```

```
print(insert_interval([[1,3],[6,9]],[2,5]))    # [[1,5],[6,9]]
```

```
print(insert_interval_inbuilt([[1,3],[6,9]],[2,5])) # [[1,5],[6,9]]
```

48. Minimum Platforms (Trains)

Without inbuilt

```
def min_platforms(arr, dep):
    arr.sort(); dep.sort()
    i=j=platforms=result=0
    while i<len(arr) and j<len(dep):
        if arr[i] <= dep[j]:
            platforms+=1; i+=1
            result = max(result, platforms)
        else:

```

```

        platforms-=1; j+=1
    return result

# With inbuilt (same but using zip)
def min_platforms_inbuilt(arr, dep):
    arr.sort(); dep.sort()
    i=j=plat=ans=0
    while i<len(arr) and j<len(dep):
        if arr[i]<=dep[j]: plat+=1; i+=1; ans=max(ans,plat)
        else: plat-=1; j+=1
    return ans

print(min_platforms([900,940,950,1100,1500,1800],[910,1200,1120,1130,1900,2000]))    # 3
print(min_platforms_inbuilt([900,940,950,1100,1500,1800],[910,1200,1120,1130,1900,2000])) # 3

```

49. Count Inversions

```

# Without inbuilt (merge sort O(n log n))
def merge_count(arr):
    if len(arr)<=1: return arr,0
    mid=len(arr)//2
    L,cL=merge_count(arr[:mid])
    R,cR=merge_count(arr[mid:])
    merged,i,j,c=[],0,0,0
    while i<len(L) and j<len(R):
        if L[i]<=R[j]: merged.append(L[i]); i+=1
        else: merged.append(R[j]); j+=1; c+=len(L)-i
    merged+=L[i:]+R[j:]
    return merged,c+cL+cR

```

```

def count_inversions(arr):
    return merge_count(arr)[1]

# With inbuilt (using sorted + bisect)
import bisect
def count_inversions_inbuilt(arr):
    seen=[]; inv=0
    for num in arr[::-1]:
        idx=bisect.bisect_left(seen,num)
        inv+=idx
        bisect.insort(seen,num)
    return inv

print(count_inversions([1,20,6,4,5]))    # 5
print(count_inversions_inbuilt([1,20,6,4,5])) # 5

```

50. Longest Consecutive Sequence

```

# Without inbuilt
def longest_consecutive(arr):
    arr_set = {}
    for num in arr: arr_set[num]=True
    max_len=0
    for num in arr:
        if num-1 not in arr_set:
            length=1
            while num+length in arr_set:
                length+=1
            max_len = max(max_len,length)
    return max_len

```

```
# With inbuilt (set trick)

def longest_consecutive_inbuilt(arr):
    nums=set(arr)
    max_len=0
    for num in nums:
        if num-1 not in nums:
            length=1
            while num+length in nums:
                length+=1
            max_len=max(max_len,length)
    return max_len

print(longest_consecutive([100,4,200,1,3,2]))    # 4
print(longest_consecutive_inbuilt([100,4,200,1,3,2])) # 4
```

Miscellaneous / Tricky

✓ Miscellaneous / Tricky Array Problems (51–60)

51. Rearrange Positive and Negative Numbers Alternately

Without inbuilt

```
def rearrange_pos_neg(arr):
    pos, neg = [], []
    for x in arr:
        if x >= 0: pos.append(x)
        else: neg.append(x)
    res, i, j = [], 0, 0
    while i < len(pos) and j < len(neg):
        res.append(pos[i]); res.append(neg[j])
        i += 1; j += 1
    res.extend(pos[i:]); res.extend(neg[j:])
    return res
```

With inbuilt (list comprehensions)

```
def rearrange_pos_neg_inbuilt(arr):
    pos = [x for x in arr if x >= 0]
    neg = [x for x in arr if x < 0]
    res = [None] * (len(pos) + len(neg))
    res[::2], res[1::2] = pos, neg
    return [x for x in res if x is not None]
```

```
print(rearrange_pos_neg([1,-2,3,-4,5,-6]))
```



```
print(rearrange_pos_neg_inbuilt([1,-2,3,-4,5,-6]))
```

52. Rearrange in Wave Form

Without inbuilt

```
def wave_form(arr):  
    n = len(arr)  
    for i in range(0,n-1,2):  
        if arr[i] < arr[i+1]:  
            arr[i],arr[i+1] = arr[i+1],arr[i]  
    return arr
```

With inbuilt

```
def wave_form_inbuilt(arr):  
    arr.sort()  
    for i in range(0,len(arr)-1,2):  
        arr[i], arr[i+1] = arr[i+1], arr[i]  
    return arr
```

```
print(wave_form([3,6,5,10,7,20]))
```

```
print(wave_form_inbuilt([3,6,5,10,7,20]))
```

53. Rearrange so that `arr[i] = i`

Without inbuilt

```
def rearrange_index(arr):  
    n = len(arr)  
    res = [-1]*n  
    for i in range(n):
```

```
    if 0 <= arr[i] < n:
        res[arr[i]] = arr[i]
    return res
```

With inbuilt

```
def rearrange_index_inbuilt(arr):
    n = len(arr)
    return [i if i in arr else -1 for i in range(n)]

print(rearrange_index([-1,-1,6,1,9,3,2,4,-1]))
print(rearrange_index_inbuilt([-1,-1,6,1,9,3,2,4,-1]))
```

54. Maximum Circular Subarray Sum

Without inbuilt (Kadane + wrap case)

```
def kadane(arr):
    max_end = max_so = arr[0]
    for x in arr[1:]:
        max_end = x if x > max_end+x else max_end+x
        max_so = max_so if max_so > max_end else max_end
    return max_so
```

```
def circular_subarray_sum(arr):
    max_kadane = kadane(arr)
    total, min_end, min_so = 0, arr[0], arr[0]
    for x in arr:
        total += x
    for x in arr[1:]:
        min_end = x if x < min_end+x else min_end+x
        min_so = min_so if min_so < min_end else min_end
```

```
if min_so == total: return max_kadane
return max(max_kadane, total - min_so)
```

With inbuilt

```
def circular_subarray_sum_inbuilt(arr):
    total = sum(arr)
    max_sum, cur, min_sum, cur_min = arr[0], arr[0], arr[0], arr[0]
    for x in arr[1:]:
        cur = max(x, cur+x); max_sum = max(max_sum, cur)
        cur_min = min(x, cur_min+x); min_sum = min(min_sum, cur_min)
    return max_sum if max_sum < 0 else max(max_sum, total-min_sum)

print(circular_subarray_sum([5,-3,5]))
print(circular_subarray_sum_inbuilt([5,-3,5]))
```

55. Leaders in an Array

Without inbuilt

```
def leaders(arr):
    n = len(arr)
    max_from_right = arr[-1]
    res = [max_from_right]
    for i in range(n-2,-1,-1):
        if arr[i] > max_from_right:
            max_from_right = arr[i]
            res.append(arr[i])
    return res[::-1]
```

With inbuilt

```
def leaders_inbuilt(arr):
```

```

res, max_so = [], float('-inf')
for x in reversed(arr):
    if x > max_so:
        res.append(x)
        max_so = x
return res[::-1]

print(leaders([16,17,4,3,5,2]))
print(leaders_inbuilt([16,17,4,3,5,2]))

```

56. Find Peak Element

Without inbuilt (binary search $O(\log n)$)

```

def peak_element(arr):
    l,r=0,len(arr)-1
    while l<r:
        mid=(l+r)//2
        if arr[mid] < arr[mid+1]:
            l=mid+1
        else:
            r=mid
    return l

```

With inbuilt (max index)

```

def peak_element_inbuilt(arr):
    return arr.index(max(arr))

print(peak_element([1,2,1,3,5,6,4]))
print(peak_element_inbuilt([1,2,1,3,5,6,4]))

```

57. Cyclically Rotate by One

Without inbuilt

```
def rotate_by_one(arr):  
    last = arr[-1]  
    for i in range(len(arr)-1,0,-1):  
        arr[i] = arr[i-1]  
    arr[0] = last  
    return arr
```

With inbuilt

```
def rotate_by_one_inbuilt(arr):  
    return [arr[-1]] + arr[:-1]
```

```
print(rotate_by_one([1,2,3,4,5]))  
print(rotate_by_one_inbuilt([1,2,3,4,5]))
```

58. Move All Zeros to End

Without inbuilt

```
def move_zeros(arr):  
    count = 0  
    for i in range(len(arr)):  
        if arr[i] != 0:  
            arr[count], arr[i] = arr[i], arr[count]  
            count += 1  
    return arr
```

With inbuilt

```
def move_zeros_inbuilt(arr):  
    return [x for x in arr if x != 0] + [0]*arr.count(0)
```

```
print(move_zeros([0,1,0,3,12]))
```

```
print(move_zeros_inbuilt([0,1,0,3,12]))
```

59. Smallest Missing Positive Integer

Without inbuilt (O(n) using index marking)

```
def first_missing_positive(arr):  
    n=len(arr)  
    for i in range(n):  
        while 1<=arr[i]<=n and arr[arr[i]-1]!=arr[i]:  
            arr[arr[i]-1], arr[i] = arr[i], arr[arr[i]-1]  
    for i in range(n):  
        if arr[i]!=i+1: return i+1  
    return n+1
```

With inbuilt (set trick)

```
def first_missing_positive_inbuilt(arr):  
    s=set(arr)  
    i=1  
    while i in s: i+=1  
    return i
```

```
print(first_missing_positive([3,4,-1,1]))
```

```
print(first_missing_positive_inbuilt([3,4,-1,1]))
```

60. Implement Array with Dynamic Memory (simulate resizing)

Without inbuilt

```
class DynamicArray:
```

```
    def __init__(self):
```

```
        self.capacity = 1
```

```
        self.size = 0
```

```
        self.arr = [None]*self.capacity
```

```
    def resize(self):
```

```
        self.capacity *= 2
```

```
        new_arr = [None]*self.capacity
```

```
        for i in range(self.size):
```

```
            new_arr[i] = self.arr[i]
```

```
        self.arr = new_arr
```

```
    def append(self, value):
```

```
        if self.size == self.capacity:
```

```
            self.resize()
```

```
        self.arr[self.size] = value
```

```
        self.size += 1
```

```
    def __str__(self):
```

```
        return str(self.arr[:self.size])
```

With inbuilt (just list)

```
class DynamicArrayInbuilt:
```

```
    def __init__(self):
```

```
        self.arr=[]
```

```
    def append(self,value):
```

```
        self.arr.append(value)
```

```
    def __str__(self):
```

```
return str(self.arr)
```

```
da = DynamicArray()
```

```
for i in range(6): da.append(i)
```

```
print(da)
```

```
db = DynamicArrayInbuilt()
```

```
for i in range(6): db.append(i)
```

```
print(db)
```


Python Arrays Problems Cheat Sheet (1–60)

◆ 1–10: Basic Array Problems

1. **Largest Element**
 - **Approach:** Scan for max.
 - **Without inbuilt:** Loop & track largest.
 - **With inbuilt:** `max(arr)`
 - **Complexity:** $O(n)$, $O(1)$
2. **Smallest Element**
 - Approach same as above with `min()`.
3. **Second Largest/Smallest**
 - Without inbuilt: Track two variables.
 - With inbuilt: `sorted(set(arr))[-2]`
 - $O(n)$ / $O(n \log n)$
4. **Reverse Array**
 - Without inbuilt: Swap ends in loop.
 - With inbuilt: `arr[::-1]`
 - $O(n)$, $O(1)$
5. **Rotate Array by k**
 - Without: Slice + concat.
 - With: `collections.deque.rotate(k)`
 - $O(n)$, $O(1)$
6. **Sum of Elements** → `sum(arr)`
7. **Average of Elements** → `sum(arr)/len(arr)`
8. **Count Even/Odd** → Loop or list comprehension
9. **Check if Sorted**
 - Without: Compare neighbors.
 - With: `arr == sorted(arr)`
10. **Merge Two Sorted Arrays**
 - Without: Merge like merge-sort.
 - With: `sorted(arr1+arr2)`

◆ 11–20: Searching & Sorting

11. **Linear Search** → Loop through elements.
12. **Binary Search (iterative & recursive)** → Divide & conquer.
13. **First & Last Occurrence** → Modified binary search.
14. **Count Occurrences** → Linear scan or `arr.count(x)`.
15. **Missing Number (1..n)** → Sum formula or XOR.
16. **Find Duplicate(s)** → Hash set / cycle detection.
17. **Pairs with Sum = k** → Hash set or two-pointer.
18. **Triplets with Sum = k** → Sort + two-pointer.

- 19. **Sorting (Bubble/Selection/Insertion)** $\rightarrow O(n^2)$.
 - 20. **Sorting (Merge/Quick)** $\rightarrow O(n \log n)$.
-

◆ 21–28: Subarray / Subsequence

- 21. **Max Subarray Sum (Kadane)** $\rightarrow O(n)$.
 - 22. **Subarray with Given Sum** $\rightarrow$ Sliding window / hashmap.
 - 23. **Longest Subarray Sum = k** $\rightarrow$ Prefix sum + hashmap.
 - 24. **Longest Equal 0s & 1s** $\rightarrow$ Prefix sum trick.
 - 25. **Max Product Subarray** $\rightarrow$ Track min & max.
 - 26. **Count Subarrays with Sum = k** $\rightarrow$ Prefix hashmap.
 - 27. **Min Length Subarray $\geq k$** $\rightarrow$ Sliding window.
 - 28. **Longest Increasing Subarray** $\rightarrow$ Linear scan.
-

◆ 29–35: Matrix (2D Array)

- 29. **Search in Sorted Matrix** $\rightarrow$ Staircase $O(m+n)$.
 - 30. **Rotate 90°** $\rightarrow$ Transpose + reverse rows.
 - 31. **Spiral Order Traversal** $\rightarrow$ Boundary pointers.
 - 32. **Transpose** $\rightarrow$ Swap indices.
 - 33. **Max Sum Submatrix** $\rightarrow$ 2D Kadane.
 - 34. **Diagonal Sum** $\rightarrow$ Sum primary + secondary diagonals.
 - 35. **Set Row/Col Zero if Element Zero** $\rightarrow$ Mark rows/cols, update.
-

◆ 36–50: Advanced Array Problems

- 36. **Majority Element ($> n/2$)** $\rightarrow$ Boyer-Moore Voting.
 - 37. **Elements $> n/3$** $\rightarrow$ Extended Boyer-Moore.
 - 38. **Equilibrium Index** $\rightarrow$ Prefix & suffix sums.
 - 39. **Trapping Rainwater** $\rightarrow$ Two-pointer.
 - 40. **Container With Most Water** $\rightarrow$ Two-pointer.
 - 41. **Max Difference ($j > i$)** $\rightarrow$ Track min element.
 - 42. **Next Greater Element** $\rightarrow$ Stack $O(n)$.
 - 43. **Next Smaller Element** $\rightarrow$ Similar stack.
 - 44. **Stock Buy/Sell Profit** $\rightarrow$ Greedy / DP (cooldown).
 - 45. **Kth Smallest/Largest** $\rightarrow$ Heap / QuickSelect.
 - 46. **Merge Intervals** $\rightarrow$ Sort + merge overlaps.
 - 47. **Insert Interval** $\rightarrow$ Merge into list.
 - 48. **Min Platforms** $\rightarrow$ Sort arrival/departure.
 - 49. **Count Inversions** $\rightarrow$ Merge sort modification.
 - 50. **Longest Consecutive Sequence** $\rightarrow$ Hash set $O(n)$.
-

◆ 51–60: Miscellaneous / Tricky

51. **Rearrange +ve & -ve Alternately** → Split & interleave.
 52. **Wave Form** → Sort + swap pairs.
 53. **Rearrange arr[i] = i** → Place correctly if exists else -1.
 54. **Circular Max Subarray** → Kadane + wrap case.
 55. **Leaders in Array** → Scan from right.
 56. **Peak Element** → Binary search.
 57. **Cyclic Rotate by One** → Shift right.
 58. **Move Zeros to End** → Two-pointer / list comp.
 59. **Smallest Missing Positive** → Index marking / set.
 60. **Dynamic Array Simulation** → Manual resize (double capacity).
-

✓ Complexity Summary

- **Linear scans (search/sum/count):** $O(n)$
 - **Sorting:** $O(n \log n)$
 - **Prefix sums / hashing:** $O(n)$ avg
 - **Two-pointer / sliding window:** $O(n)$
 - **Divide & conquer (merge sort, quick sort, inversions):** $O(n \log n)$
 - **Matrix problems:** $O(m \cdot n)$ or $O(n^3)$ for submatrix sum
-

?